

# Gestione della memoria

La **memoria principale** (*RAM*) è una componente fondamentale, che cresce rapidamente, ma non alla stessa velocità dei programmi. Il desiderio è che sia privata, veloce, persistente e a basso costo, ma nella realtà tecnologica la situazione è diversa.

Il concetto di *gerarchia della memoria* si è sviluppato nel tempo per ridurne i costi, ed il gestore della memoria gestisce anche la sua gerarchia, traccia l'uso della memoria e ne alloca o libera spazio per i processi.

Il modello più semplice è l'utilizzo diretto della memoria fisica in monoprogrammazione. Nasce poi la possibilità di eseguire più programmi contemporaneamente senza astrazione della memoria utilizzando lo **swapping** (salvataggio del contenuto *della memoria in un file su memoria non volatile* e prelievo del programma successivo ).

## Astrazione della memoria

L'accesso diretto alla memoria fisica da parte dei programmi può causare problemi come la distruzione del SO e la difficoltà di esecuzione simultanea di più processi. La **soluzione** è l'astrazione della memoria per separare e proteggere i programmi in esecuzione. Nasce il concetto di *spazio degli indirizzi*:

- Ogni programma ha un *insieme unico di indirizzi* che può usare per indirizzare la memoria.
- Questo spazio è indipendente da altri processi e rappresenta una forma di memoria astratta.

Per la gestione del sovraccarico di memoria si utilizzano

- **Swapping dei processi:**
  - Sposta interi processi tra memoria RAM e memoria non volatile (disco/SSD);
  - Processi inattivi archiviati su memoria non volatile.
- **Memoria virtuale:**
  - Permette l'esecuzione dei programmi anche se *solo parzialmente presenti nella memoria principale*.

## Gestione della memoria libera

Per tenere traccia dell'utilizzo della memoria esistono due metodi principali:

- **Bitmap** che tiene traccia di quali blocchi vengono allocati;
- **Liste concatenate** che tengono traccia della memoria non allocata.

Vengono spesso scelte le liste concatenate, poiché trovare i blocchi non allocati richiede una scansione lenta. Nella pratica viene spesso usata *una doppia linked list*, che rende più facile gestire lo spazio libero.

Schemi di allocazione della memoria:

- **First Fit**: Seleziona il primo spazio disponibile (più semplice)
- **Next Fit**: Seleziona il successivo spazio disponibile (più lento)
- **Best Fit**: Seleziona lo spazio più adeguato (porta a frammentazione)
- **Worst Fit**: Seleziona lo spazio meno adeguato (prestazioni scadenti)
- **Quick Fit**: Mantiene spazi di dimensioni diverse (scarsa performance nella coalescenza)
- **Buddy Allocation** (Linux): Migliora la coalescenza del QF. La memoria inizia come un singolo pezzo contiguo, che ad ogni richiesta viene divisa secondo una potenza di 2. Quando la memoria viene rilasciata i blocchi contigui si riuniscono. L'unico problema è che se lo spazio richiesto non è in potenza di 2 l'algoritmo porta ad una frammentazione interna.

- **SLAB Allocator** prende blocchi tramite l'algoritmo e ritaglia unità più piccole (*slab*) per gestirle separatamente.

Nello **SLAB ALLOCATION** la memoria è divisa in blocchi chiamati *slabs* (ulteriormente suddivisi in chunk di dimensioni uniformi). Uno slab può essere in uno dei seguenti stati:

- **Pieno** (tutti i chunk utilizzati);
- **Parzialmente pieno** (alcuni chunk liberi);
- **Vuoto** (tutti i chunk liberi).

Quando un oggetto viene deallocato non viene immediatamente restituito al sistema come memoria libera, ma *mantenuto nella cache* in modo che, se viene richiesta un'altra istanza dello stesso tipo di oggetto, possa essere riallocata.

## Memoria virtuale

La memoria virtuale *estende* l'idea di registri base e limite. Ogni programma ha un proprio spazio degli indirizzi suddiviso in "*pagine*", che sono intervalli di indirizzi

contigui. Non tutte le pagine devono essere contemporaneamente in memoria fisica, poiché l'HW crea una mappa di quelle direttamente in memoria, e se qualcuna manca, il SO interviene.

La maggior parte dei sistemi moderni usa il "*paging*" (paginazione), ovvero la divisione dello spazio degli indirizzi in unità di dimensione fissa.

L'alternativa è la *segmentazione* con unità di dimensione variabile, ma ormai meno comune.

L'idea è quella di creare per il processo l'illusione di uno spazio di indirizzi ampio, lo **spazio di indirizzi virtuale**. La RAM è nota come *memoria fisica*, mentre **Memory Management Unit (MMU)** traduce gli indirizzi virtuali in indirizzi fisici. I sistemi moderni utilizzano la *paginazione* dividendo la memoria fisica e virtuale in pagine di dimensioni fisse, e traducendo le pagine virtuali in *pagine fisiche* (frame).

Se un programma fa riferimento ad una pagina non mappata, si verifica un **Page Fault**. Il SO allora gli assegna un frame, spostando eventuali frame raramente usati se serve, con uno specifico criterio. Quindi poi inizializza/carica la pagina richiesta nel frame libero o liberato, ed aggiorna la mappa della MMU per riflettere i cambiamenti.

La *relazione* tra gli indirizzi di memoria virtuale e fisica è data dalla **Page Table**.

Le voci di una Page Table sono composti da:

- *Bit presente/assente* - indica se la pagina virtuale è in memoria.
- *Bit protezione* - specifica i tipi di accesso consentiti (rwx).
- *Bit supervisor* - stabilisce se la pagina è accessibile solo al SO o anche ai programmi utente
- *Bit Modificato e Riferimento* - registrano l'uso della pagina.

Per velocizzare la paginazione bisogna decidere dove memorizzarla:

- Nei **Registri HW**: Un registro HW per ogni pagina virtuale, caricato all'avvio del processo. È semplice e non richiede accessi alla memoria durante la mappatura, ma risulta costoso con tabelle di pagine grandi.
- Nella **Memoria Principale**: Tabella interamente in RAM, con un registro che punta al suo inizio. Facile da cambiare ad ogni cambio di contesto, e richiede solo il caricamento di un registro, ma richiede anche accessi frequenti alla memoria rendendo la mappatura più lenta.

Si introduce il **Translation Lookaside Buffer (TLB)**, un dispositivo HW che mappa gli indirizzi virtuali in fisici senza passare per la tabella delle pagine, riducendo gli accessi alla memoria durante la paginazione.

Il TLB è strutturato come un piccolo numero di voci, ciascuna con numero di pagina virtuale, bit modificato, codice di protezione e frame fisico. Alla richiesta di un indirizzo virtuale l'MMU consulta prima il TLB. Se trovato e valido (*TLB HIT*) il frame è prelevato direttamente dal TLB, altrimenti (*TLB MISS*) avviene una ricerca normale nella tabella delle pagine e la voce trovata ne rimpiazza un'altra nel TLB.

I TLB MISS sono *comuni* a causa del numero limitato di voci nel TLB, al quale però non avrebbe senso aumentare la dimensione.

- *Soft Miss*: La pagina è in memoria ma non nel TLB.
- *Hard Miss*: La pagina non è neanche in memoria e richiede l'accesso al disco

## Algoritmi di Sostituzione delle pagine

Quando una pagina logica non è in memoria il SO deve caricarla in caso di page fault. Quindi un'altra pagina logica potrebbe essere scambiata, ma quale?

### Algoritmo Ottimale:

- Il *concetto* è scegliere la pagina con il riferimento più distante nel futuro da rimuovere, per rimuovere *idealmente* la pagina che non sarà utilizzata per il maggior numero di istruzioni future.
- *Problema*: È impossibile per il SO prevedere il momento del prossimo riferimento per ciascuna pagina.

Ricordiamo: Bit della tabella delle pagine per gli algoritmi di sostituzione:

- *Modified* (M): Impostato quando una pagina viene modificata;
- *Referenced* (R): Impostato quando la pagina viene acceduta.

### Not Recently Used(NRU):

- L'*obiettivo* è trovare le pagine non modificate che non sono state accedute "recentemente" utilizzando i bit di stato R e M.
- I bit vengono impostati dall'HW ad ogni accesso, ed il bit R viene periodicamente ripulito per identificare pagine non recentemente utilizzate.
- Le pagine vengono classificate in 4 classi (da 0 a 3) in funzione dell'uso e delle modifiche:
  - *CO*: Non referenziata, non modificata
  - *CI*: Non referenziata, modificata

- **C2**: Referenziata, non modificata
- **C3**: Referenziata, modificata
- NRU rimuove una pagina casuale della classe più bassa non vuota. L'algoritmo è semplice, efficiente ed offre prestazioni accettabili.

### First-In, First-Out (**FIFO**):

- Elimina la pagina **più vecchia** in memoria. Il SO rimuove la pagina in testa alla lista durante un Page Fault, aggiungendo la nuova pagina in coda.
- Raramente utilizzato nella sua forma più semplice, poiché la pagina più vecchia potrebbe ancora essere frequentemente utilizzata.

### Seconda Chance:

- Principalmente si controlla il bit R della pagina più vecchia per decidere la rimozione. Se  $R=0$  la pagina è vecchia e non usata di recente, quindi viene sostituita; se  $R=1$  il bit viene azzerato e la pagina viene reinserita in fondo alla lista.

### Clock:

- Lista circolare dei frame di pagina con un puntatore che fa da "lancetta" di un orologio per identificare la pagina più vecchia.
- Ad un page fault se il bit R della pagina puntata è 0, la pagina viene rimossa e sostituita con la nuova, poi la lancetta avanza. Se  $R=1$  il bit viene azzerato ed il puntatore si sposta alla pagina successiva. Il concetto di base è ripetere il processo finché non trova una pagina con  $R=0$ .
- Elimina l'inefficienza della continua riallocazione delle pagine lungo la lista. È più performante rispetto a FIFO e Seconda Chance.

### Least Recently Used (**LRU**):

- **IN TEORIA** le pagine non usate di recente sono candidate alla sostituzione. La possibile implementazione è una lista delle pagine con quelle più utilizzate in testa e meno utilizzate in coda. Ogni riferimento richiede l'aggiornamento della lista e copia di pagine intere (troppo costoso).
- Sebbene tendente all'ottimo, praticamente non efficiente e non utilizzato

### Not Frequently Used(**NFU**):

- Associa un contatore ad ogni pagina, incrementato con ogni interrupt del clock in base al bit R - Tanti accessi -> Alto valore di "frequenza" assegnato alla pagina -> minore possibilità di rimozione.
- Non dimenticando il passato, potrebbe non sostituire una pagina che in un determinato periodo era molto utilizzata ma adesso non lo è più
- Miglioramento : *Aging*
  - Numero di bit fisso
  - Ad ogni interrupt del clock i bit vengono spostati a destra
  - Prima dello shift dei contatori il bit R viene aggiunto al lato sinistro
- Emula LRU, dando meno peso agli usi passati e preferendo le pagine meno referenziate di recente.
- Non distingue l'ordine esatto dei riferimenti recenti ed ha un orizzonte temporale limitato (non per forza un male)

## Concetto di Working Set

Per **Working Set** si intende l'*insieme delle pagine attualmente usate da un processo*.

Rappresenta le pagine a cui un processo fa riferimento durante la fase dell'esecuzione.

- **Demand Paging**: Le pagine sono caricate in memoria "a richiesta", solo quando necessario.
- Inizialmente si verificano molti PageFault, finché non vengono caricate tutte le pagine necessarie.

Se il Working Set di un processo è *completamente in memoria* si verificano pochissimi Page Fault, ma se il WS è *più grande* della memoria disponibile, si verificheranno troppi PF (fenomeno noto come *thrashing*).

### Working Set Model:

- Molti SO cercano di *tracciare il WS* di ogni processo e di mantenerlo in memoria per ridurre i PF;
- La *pre-paginazione* carica in anticipo le pagine basandosi sul WS del processo. Il WS è spesso definito in termini di *tempo*, ovvero le pagine usate negli ultimi  $\tau$  secondi di esecuzione.

### Algoritmo di sostituzione basato sul Working Set:

- Alla verifica di un PF, si ricerca una pagina fuori dal WorkingSet per rimuoverla

- Utilizza informazioni come il bit  $R$  ed il tempo dell'ultimo utilizzo per determinare quali pagine rimuovere. Durante un PF:
  - Scansiona le pagine alla ricerca di una pagina da rimuovere
  - *Controllo del bit  $R$  per ogni pagina:*
    - $R = 1$ : Aggiornamento del tempo dell'ultimo utilizzo, la pagina è nel WS
    - $R = 0, Et\grave{a} > \tau$ : La pagina non è nel WS e viene rimossa
    - $R = 0, Et\grave{a} < \tau$ : La pagina rimane ma si contrassegna come la più vecchia per la possibile rimozione.
  - Se nessuna pagina è rimovibile, viene selezionata la più vecchia con  $R = 0$ 
    - Altrimenti una a caso.

### Algoritmo WSClock:

- Miglioramento dell'algoritmo Clock che integra le info del WS. Semplice e con buone prestazioni;
- Struttura dati:
  - Lista circolare di frame (simile al Clock)
  - Ogni frame nella lista contiene:
    - Tempo dell'ultimo utilizzo
    - Bit  $M$
    - Bit  $R$
- Ad ogni PF viene esaminata per prima la pagina indicata dalla lancetta. Se  $R = 1$  la pagina *non è la candidata ideale* alla rimozione. Il ciclo viene quindi impostato a 0
- La lancetta avanza alla pagina successiva e l'algoritmo viene ripetuto per la nuova pagina. Se la pagina indicata ha  $R = 0$  e l'età è maggiore di  $\tau$ :
  - Se  $M = 0$  (pagina pulita):
    - Non è nel WS e ne esiste una copia valida su memoria non volatile
    - Il frame viene riciclato e vi viene posta la nuova pagina
  - Se  $M = 1$  la pagina è sporca:
    - NON ne esiste una copia valida in memoria non volatile
    - Non può essere rimossa immediatamente
- Per evitare rallentamenti la scrittura su memoria non volatile viene schedulata e rimandata

# Problemi di progettazione

La paginazione è un processo complesso che richiede una comprensione approfondita di molteplici aspetti per una progettazione efficace, come l'*allocazione della memoria*, la *gestione dei PF*, l'*ottimizzazione delle prestazioni* e le *decisioni di progettazione*.

## Allocazione di memoria

### Allocazione locale:

- Ogni processo riceve una porzione fissa della memoria;
- Semplice da implementare ma può portare a inefficienze.

### Allocazione globale:

- Distribuzione dinamica della memoria tra i processi;
- Più efficace per adattarsi alle esigenze variabili dei processi, ma richiede una gestione più complessa.

Vantaggi:

- *Adattabilità degli algoritmi globali* per aumentare l'efficienza quando la dimensione del WS varia nel tempo
- *Limite degli algoritmi locali* come il thrashing o la memoria sprecata
- *Gestione dinamica della memoria* tramite i bit di Aging per monitorare la frequenza di accesso alle pagine

## Strategie

### Allocazione Equa:

- Distribuzione uniforme dei frame tra processi, non tiene conto delle diverse esigenze di memoria tra processi di dimensioni varie.

### Allocazione Proporzionale:

- Assegnazione di frame in base alla dimensione del processo
- Rispecchia meglio le necessità di memoria evitando allocazioni inadeguate

### Importanza del Limite Minimo di Pagine:

- Assicurare che *ogni processo abbia abbastanza pagine* per eseguire le operazioni fondamentali

- *MA prevenire situazioni* in cui i processi con istruzioni che *attraversano i limiti* delle pagine non possano eseguire.

## Dinamica di allocazione e algoritmo PFF

### Gestione dinamica dei frame:

- Si inizia con un'allocazione proporzionale alla dimensione del processo
- Aggiornamento dinamico dell'allocazione in base all'evoluzione delle esigenze durante l'esecuzione

#### Page Fault Frequency:

- *Monitoraggio della frequenza di PF per regolare l'allocazione di memoria di un processo*
- Aumenta i frame se PF troppo frequenti, diminuisce se sono rari.

Anche con il miglior algoritmo, il *thrashing* può purtroppo sempre verificarsi se i WS di tutti i processi eccedono la memoria disponibile. Il PFF può segnalare una richiesta collettiva di più memoria senza che nessun processo possa cedere frame.

### Mitigazione:

- *Out Of Memory Killer*: Processo di sistema che seleziona e termina i processi ad un punteggio di "cattiveria" per liberare memoria (elevato utilizzo o minor importanza)
- *Swapping*: Sposta i processi su memoria non volatile, liberando le loro pagine per altri processi. Può ridurre la richiesta di memoria senza interrompere l'esecuzione dei processi

### Tecniche di riduzione di memoria

- *Scheduling a due livelli*:
  - Alcuni processi sono in memoria non volatile e solo una parte è schedata attivamente
  - Aiuta a gestire meglio il carico di memoria
  - Utile per ridurre occupazione di memoria di processi in background in sistemi interattivi
- *Gestione della multiprogrammazione*:
  - La selezione dei processi da spostare considera anche altre caratteristiche, come la dimensione e/o frequenza di paginazione dei processi e se sono CPU-Bound o I/O-bound

## Dimensione delle pagine

I SO possono selezionare la dimensione delle pagine.

- *Pagine piccole*: Riducono la frammentazione interna e l'utilizzo della memoria, ma richiedono tabelle delle pagine più grandi e possono aumentare tempo e spazio necessari per trasferimento dati e gestione di memoria.

La **dimensione ottimale** viene determinata equilibrando frammentazione interna e overhead della tabella delle pagine. Alcuni SO utilizzano pagine di diverse dimensioni per parti diverse del sistema.

*Parametri considerati:*

- Dimensione media del processo:  $s$  byte
- Dimensione della pagina:  $p$  byte (da calcolare)
- Dimensione di ogni voce nella tabella delle pagine:  $e$  byte

*Calcolo Overhead:*

- Numero di pagine per processo:  $\approx s/p$
- Spazio occupato nella tabella delle pagine:  $s \cdot e/p$  bytes
- Memoria sprecata per frammentazione interna nell'ultima pagina:  $p/2$ 
  - *Fenomeno dell'ultima pagina*: Per qualsiasi processo l'ultima pagina di memoria allocata potrebbe non essere completamente riempita.

**Overhead Totale:**  $\frac{s \cdot e}{p} + \frac{p}{2}$ :

- Il primo termine aumenta con pagine più piccole
- Il secondo aumenta con pagine più grandi
- L'ottimo si trova *bilanciando questi due fattori*

## Condivisione delle pagine

È comune che molti utenti eseguano lo stesso programma o utilizzino le stesse librerie. Condividere pagine di memoria tra questi processi è più efficiente che mantenerne copie separate. Le *pagine di sola lettura* sono spesso *condivisibili*, mentre le *pagine di dati* generalmente *no*. È opportuno separare spazi di indirizzo in:

- *I-Space*: Istruzioni;
- *D-Space*: Dati.

Processi *diversi* possono utilizzare la *stessa tabella delle pagine per l'I-Space* ma diverse per il D-Space.

Tuttavia, la *rimozione di un processo* dalla memoria può causare numerosi PF in un altro processo che condivide le stesse pagine, quindi è cruciale sapere se le

pagine sono ancora in uso per evitare la loro liberazione accidentale.

**Condivisione dei dati:** Più complessa rispetto alla condivisione del codice.

**Copy On Write** (*Copia in caso di scrittura*): Se un processo modifica i dati, si genera una trap, e viene creata una copia della pagina modificata. Entrambe le copie diventano poi modificabili, ma *si evita la copia di pagine che non vengono mai modificate*.

## File Mappati in memoria

**Concetto:** I file mappati in memoria consentono ad un processo di mappare un file all'interno del proprio spazio di indirizzi virtuali. Alla mappatura, *nessuna pagina viene caricata immediatamente*. Quando *il processo termina* o la mappatura è eliminata, *tutte le pagine modificate vengono riscritte sul file*.

- **Modello I/O alternativo:** Offre un modo diverso di eseguire I/O, permettendo di accedere al file come se fosse un grande array di caratteri in memoria.

**Comunicazione tra processi:** Se più processi mappano lo stesso file contemporaneamente, possono comunicare attraverso questa memoria condivisa

## Gestione dei Page Fault

**Gestione dei Page Fault:**

- *Determinare l'indirizzo virtuale* che ha causato il *fault*;
- *Trovare la pagina necessaria* nella memoria non volatile;
- *Scegliere un frame disponibile* eventualmente rimuovendo pagine vecchie;
- *Caricare la pagina nel frame* e ripristinare il contatore del programma.

**Chiusura del processo:**

- *Rilasciare la tabella delle pagine*, le pagine in memoria e lo spazio su disco;
- *Gestire le pagine condivise con altri processo*, rilasciandole solo dopo l'ultimo utilizzo.

## Page Fault in 10 passi

### A. Inizio della sequenza

#### 1. Trap nel Kernel da parte dell'HW

- L'HW esegue una trap nel kernel, salvando il contatore del programma nello stack
- Informazioni sull'istruzione corrente salvate nei registri speciali della CPU

## 2. *Avvio dell'Interrupt Service Routine*

- Viene eseguita una routine in assembly per salvare i registri e altre informazioni non volatili
- Invocazione del gestore dei Page Fault

## 3. *Identificazione della pagina virtuale necessaria*

- Il SO determina quale pagina virtuale manca
- Se non disponibile dai registri HW, recupero e analisi dell'istruzione dal contatore del programma

### B. Gestione e risoluzione

## 4. *Verifica validità indirizzo e protezione*

- Controllo della validità dell'indirizzo e coerenza della protezione con l'accesso
- Se invalide, invio di un segnale di errore o terminazione del processo

## 5. *Rilascio di un frame libero*

- Se non ci sono frame liberi, esecuzione di un algoritmo di sostituzione delle pagine
- Se la pagina è sporca viene schedulata per la scrittura in memoria non volatile ed il processo viene sospeso

## 6. *Caricamento pagina richiesta*

- Frame libero usato per caricare la pagina necessaria da disco
- Durante il caricamento, il processo in PF è ancora sospeso e viene eseguito, se disponibile, un altro processo

### C. Conclusione e ripresa

## 7. *Aggiornamento delle tabelle delle pagine*

- Le tabelle delle pagine vengono aggiornate per riflettere la nuova posizione della pagina
- Il frame viene contrassegnato come disponibile

## 8. *Ripristino dell'istruzione in errore*

- Istruzione riportata allo stato che aveva all'inizio
- Il contatore del programma viene ripristinato in modo da puntare a quell'istruzione

## 9. *Ripresa del processo in errore*

- Processo rischedulato per l'esecuzione

- Ritorno alla routine in Assembly che lo aveva interrotto

#### 10. *Ricarica dei registri e ritorno allo spazio utente*

- L'ISR ricarica i registri e le info di stato
- Il controllo ritorna allo spazio utente per continuare l'esecuzione da dove era stata interrotta

## Segmentazione

Si tratta di un'ulteriore tecnica per la gestione della memoria, che suddivide la memoria in *segmenti logici distinti*. Ciascun segmento ha una sequenza lineare di indirizzi, che possono avere lunghezze diverse e variabili durante l'esecuzione. Questa struttura consente ai segmenti di *crescere o ridursi senza interferire l'uno con l'altro*. Per specificare un indirizzo in memoria si usa un indirizzo a due parti:

- Numero di segmento
- Indirizzo nel segmento

#### Vantaggi:

- *Flessibilità*: I segmenti possono crescere o ridursi in modo indipendente l'uno dall'altro
- *Semplificazione del linking*: Se ogni procedura occupa un segmento separato, il linking di procedure diventa molto più semplice poiché in caso di modifiche non è necessario aggiornare gli indirizzi di altre procedure non correlate
- *Condivisione e protezione*: La segmentazione *facilita la condivisione di risorse* tra processi diversi